# Orbital Dragons to Hunt Cryptids: Utilizing Non-Euclidean Geometry to Solve Busy Beaver

Enan Srivastava

DeepMath 2026 — Extended Abstract

**Abstract**

The busy beaver value $BB(n)$ is the longest finite runtime of any $n$-state Turing machine started on a blank tape. $BB(5)$ was settled in 2024 with a computer-checked proof. $BB(6)$ is blocked by a handful of machines whose tapes secretly run Collatz-style arithmetic; nobody can prove whether they stop. We study two of these machines. Our approach is empirical first: we log the machine's behavior as a sequence of events, embed the event data in logarithmic, simplex, and hyperbolic coordinates, and find that the dynamics are piecewise affine in a five-integer state. We then prove this description correct in the Lean proof assistant, down to the machine's individual steps and with a minimal axiom footprint. The result is a machine-checked theorem: if a simple five-integer recurrence never enters an explicit finite set of bad configurations, the machine never halts. We also prove the converse pressure: entering a bad configuration forces the machine to halt. The open problem is thereby moved, without loss, from a Turing machine to a short arithmetic recurrence. We then derive the recurrence's renewal map in closed form, show its fatal set is a width-five window around the diagonal, and close the two mechanical proof routes with computational evidence: no congruence invariant up to modulus 384 excludes the window, and the branch dynamics has exponential factor complexity, so it is not conjugate to a rotation. Measuring the recurrence over its whole seed space shows the set of dying seeds has natural density zero at the Borel–Cantelli rate ($\approx 12/X$), so our machines' seeds are typical survivors. We frame the program on a verification spectrum whose proven endpoint is $\mathsf{MIP}^* = \mathsf{RE}$: kernel checking at the bottom, certified predictive emulation — this paper — in the middle, and entangled-prover verification of halting-type claims as the goal. Every empirical claim is validated against $10^9$ raw machine steps, and the recurrence is tested for $10^8$ iterations.

## 1 The problem, from zero

A Turing machine here is a finite table: six control states, a tape of 0s and 1s, and for each (state, read symbol) pair an instruction — write a bit, move left or right, change state — or nothing. If the table has no instruction for the current situation, the machine halts. $BB(n)$ asks: among all $n$-state tables that eventually halt from the all-zero tape, what is the longest runtime? The function is noncomputable, so each new value is won machine by machine: every table either halts (run it) or must be *proven* to run forever.

The bbchallenge collaboration settled $BB(5) = 47{,}176{,}870$ with a proof checked in the Rocq proof assistant [1, 2]. At six states a residual set of machines defeats all known automated deciders. The community calls them cryptids. The reason they are hard is concrete: their tape dynamics implement iterated arithmetic maps of the same character as the Collatz $(3x+1)$ map — deterministic integer maps where parity decides the next move, and where the question is whether one

specific orbit ever hits a small bad set. The strongest known result for such maps is Tao's: *almost all* Collatz orbits reach almost bounded values [3]. That theorem is about typical orbits. Busy beaver questions need the worst case: one orbit, followed forever. No general tool exists for that, which is why each cryptid is studied on its own terms.

We work on two six-state cryptids, a mirror-image pair we call dossier-2 and dossier-3. Machine one, written `1RB1RC_1LC1LC_1RE0LD_1LB1RF_---1RA_0RD1RD` in the standard notation, is the following table. Each entry gives (bit written, head move, next state); the machine starts in state $A$ on an all-zero tape.

| state | reads 0 | reads 1 |
|---|---|---|
| $A$ | 1, R, $B$ | 1, R, $C$ |
| $B$ | 1, L, $C$ | 1, L, $C$ |
| $C$ | 1, R, $E$ | 0, L, $D$ |
| $D$ | 1, L, $B$ | 1, R, $F$ |
| $E$ | **(none: halt)** | 1, R, $A$ |
| $F$ | 0, R, $D$ | 1, R, $D$ |

The single empty entry is the only way to halt: the machine must be in state $E$ reading a 0. Working backward through the table, state $E$ is only entered from state $C$ reading a 0, so any halt must pass through the configuration "$C$ reads 0 with another 0 to its right." We call this the *probe event*. Everything that follows is one long argument that the probe event never happens.

## 2 What we do

Our method has three stages. Watch, compress, certify.

**Watch.** We run the machine and record only the moments that matter. For these machines there is a natural clock: a recurring configuration we call a spawn entry. Between two spawn entries the machine performs one bounded "episode" of work. Sampling at spawn entries turns a raw trajectory of $10^9$ steps into a clean sequence of a few million events.

**Compress.** At every spawn entry the tape has the same form: a solid block of 1s with exactly two 0-cells ("holes") inside it, the head at the block's right end. A concrete instant of the certified orbit (machine at position $r$, dots are 0s):

$$\cdots 0\,0\,\underbrace{1\,1\,1\,1\,1\,1}_{a}\,\underset{h_1}{0}\,\underbrace{1\,1\,1\,1}_{b-1}\,\underset{h_2}{0}\,\underbrace{1\,1\,1\,1\,1\,1\,1}_{c-1}\,\underset{r}{\downarrow}\,0\,0\cdots$$

So the whole configuration is five integers: the block's endpoints $l, r$, the hole positions $h_1, h_2$, and a tag; we write $a = h_1 - l$, $b = h_2 - h_1$, $c = r - h_2$ for the gaps. Between spawn entries the machine performs one of seven "episodes," each later certified with its exact step count:

| episode | steps | effect on the tape |
|---|---|---|
| spawn | 7 | extend block by 1, dig a fresh hole at $r-2$ |
| crawl | 10 per cycle | the walking gait; moves 2 cells, carries a hole |
| cross | 13 | absorb a hole, re-dig it 4 cells right |
| terminate | 13 | fill a hole, plant it 6 right (the probe events) |
| fuse | 19 | merge adjacent holes, plant 9 right |
| edge | 10 or 13 | extend the left end by 1 or 3 (renewal) |
| sweep | 2 per cell | return home; holes cross for free |

For instance, the crawl is a fixed ten-step loop: the machine marches left three cells twice (writing $1, 1, 0$ each time), then backs up four; the net effect is two cells of travel and a 0 that the walker

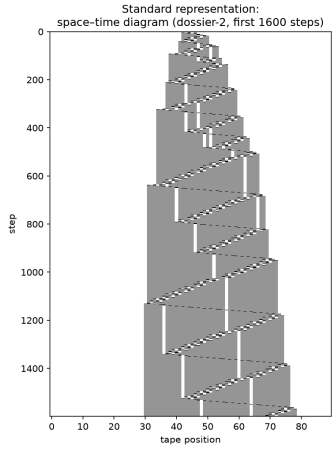



Figure 1: Raw view: the machine's first 1,600 steps (dark = 1s, one row per step). Nothing about the long-run law is visible here.

carries along three cells behind itself. The danger state $C$ reads a 0 *only* inside terminate and fuse episodes, and there the theorems show the cell to its right always holds a 1 — the probe event cannot fire. Plotted in raw coordinates, the event data looks like growth curves with no visible law (Figure 1). Plotted in the right coordinates the same data is piecewise linear: componentwise $\log(1+x)$ straightens each regime, the projective simplex $(a, b, c)/(a+b+c)$ removes scale, and hyperbolic and spherical renderings of those two charts expose the orbit's nested renewal structure (Figure 2). Straight segments are one regime; corners are regime changes. These coordinate choices are standard elsewhere: the log map is the tropical limit of algebraic geometry — historically, a dequantization, the $\hbar \to 0$ limit of a quantum calculus [6, 7]; the simplex normalization with its Fisher–Rao metric is the classical face of quantum state geometry, made literal by the amplitude map $p \mapsto \sqrt{p}$; and shape-versus-scale is the hyperbolic geometry used for hierarchical data [8]. The underlying view of a Turing machine as a dynamical system goes back to Moore and Kůrka [4, 5].

**Certify.** The picture suggests a law; the law must then be proven about the actual machine, or it is only a picture. We first validate it empirically and fail-closed: a predictor that knows the conjectured law must reproduce every event — exact positions, exact tape — over $10^9$ raw steps per machine, and any miss aborts. It reproduces all of them. We then prove the law in Lean 4, using only the kernel's minimal axioms (propext, Quot.sound; no classical choice). The certified account consists of: seven theorems, one per episode type, each giving the episode's exact step count, exact output shape, and safety; two inductions that handle the episodes' unbounded middle parts; a concrete certificate for the machine's startup (235 steps for one machine, 2,044 for the other); and a transport theorem showing that a five-integer function metaStep, iterated from the startup state, predicts the machine forever.

## 3 The result

Halting, for these machines, can only happen one way: a specific control state must read a 0 with another 0 beside it. The certified episode theorems show this situation never arises while the five-integer iteration stays inside an explicit guarded region. Writing SafeMetaOrbit for the statement that the iteration never leaves that region:

**Theorem 1.** *For each machine, checked by the Lean kernel at axioms* {propext, Quot.sound}*:*

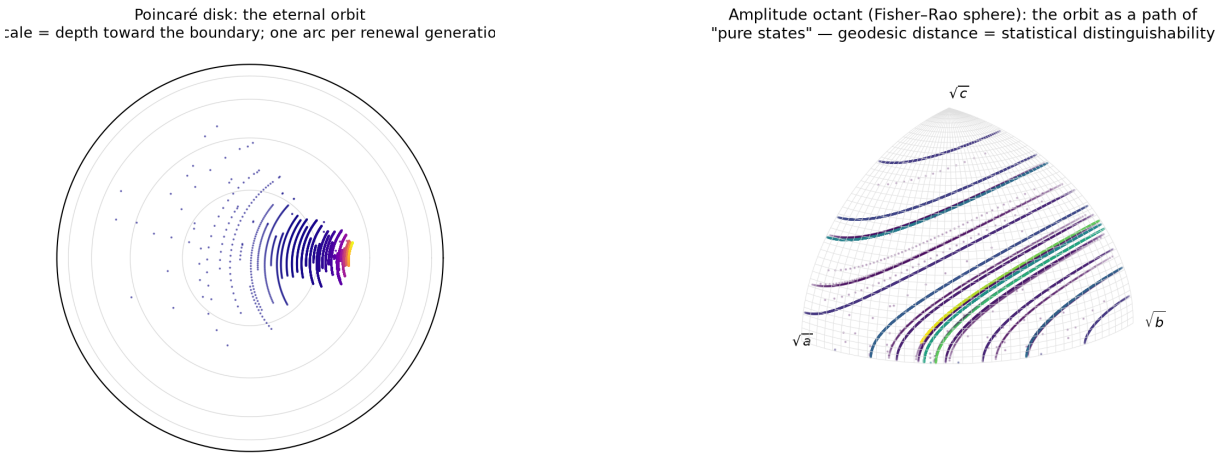



Figure 2: The same machine, 100,050 events, in two non-Euclidean coordinate systems. Left: the Poincaré disk — shape runs along the boundary direction, log-scale is hyperbolic depth, and each renewal generation of the orbit sits one nested arc deeper. Right: the amplitude octant — the normalized state $(a, b, c)/(a{+}b{+}c)$ mapped by $p \mapsto \sqrt{p}$ onto the Fisher–Rao sphere, so the orbit becomes a path whose geodesic length measures statistical distinguishability. Straight arcs are single regimes; corners are regime changes.

$\mathsf{SafeMetaOrbit} \to \neg\mathsf{RawHalts}(\mathsf{blankStart})$.

Two facts make this reduction meaningful rather than cosmetic. First, it is complete: startup, one exceptional episode, and every future step of the machine are covered by theorems, so nothing about the machine remains to be understood — only the arithmetic. Second, it is sharp: we constructed the configurations just outside the guarded region and ran the machine from them; it halts within about fifty steps. Leaving the region is not merely unproven-safe. It is fatal. So SafeMetaOrbit *is* the halting problem, restated as arithmetic.

What kind of arithmetic? Inside the main regime the iteration is $(a, b, c) \mapsto (a{+}6,\ b{-}2,\ c{-}2)$, so $\gamma = c - b$ is conserved while the counters drain. When $b$ or $c$ runs out, a renewal move re-seeds the triple by subtracting and swapping counters, with parity choosing the branch — part Euclidean algorithm, part Collatz. Figure 3 shows the renewal process as its own dynamical system: the return map of $\gamma$, with the fatal values $\gamma \in \{1, 3, 5\}$ marked. The recurrence avoids the band for $10^8$ iterations (about $10^{15}$ machine steps). Proving it does so forever is the open problem, now in a form a number theorist can hold in one hand.

We characterized that problem further. Sampling the orbit at its renewal section reduces the whole dynamics to a two-counter map, verified exactly against the certified orbit: with $(\alpha, \beta) = (6, a{-}1)$ or $(9, a{-}2)$ by the parity of $a$,

$$R(a,b) = \begin{cases} (a+6,\ b-3) & b \text{ even}, \\ (\alpha+3(b-3),\ \beta-(b-3)) & b \text{ odd},\ b<\beta, \\ (\alpha+3\beta+3,\ b-\beta-4) & b \text{ odd},\ b>\beta. \end{cases}$$

Its fatal set is geometrically simple: death occurs exactly when the two counters arrive almost equal ($b - \beta \in \{1, 3, 5\}$, plus two boundary cases). Two mechanical proof routes are then closed by computation. A sound residue-abstraction search finds a genuine mod-3 conservation law but

shows that no congruence invariant up to modulus 384 excludes the fatal window. And the branch-symbol sequence has factor complexity growing like $1.6^n$ — a rotation coding would give $n+1$ — so the dynamics is not conjugate to a rigid system, and continued-fraction rigidity arguments do not apply as stated. Over $10^5$ renewals the orbit's minimum distance above the fatal window is 4: it grazes death and survives, so no proof can rely on a structural buffer. What remains is a pointwise avoidance statement for a pseudorandom multiplicative map — the Collatz situation — with three mitigations: the map is explicit and total, the fatal set is a finite union of lines near the diagonal, and the certified reduction stands under any future argument. The natural next language is hyperbolic: renewal steps are integer $2\times2$ matrix moves, and a near-tie of the counters is a cusp excursion on the modular surface, so the conjecture asks that a specific geodesic avoid three explicit horoballs forever [9].

*Almost all seeds survive.* We also measured the map $R$ over its whole seed space, not just our orbit. Enumerating every seed in $[X, 2X)^2$ exactly for $X \le 256$, and sampling to $X = 2^{16}$, the fraction of seeds whose orbit ever dies is $\approx 12/X$: the dying set has natural density zero, at the rate a Borel–Cantelli heuristic predicts. The mechanism is visible in the data. Every observed death occurs within the first fifty renewals; the offset $b-\beta$ at the diagonal is uniformly distributed, fatal values included; and sizes grow by a factor $e^{0.134}$ per renewal, so the per-renewal hazard $\approx 6/\text{size}$ sums geometrically — death is an early-game phenomenon, and survival compounds. Our machines' seeds are therefore *typical survivors*, not exceptions. (One measurement artifact is worth recording: an early instrument logged offsets only on steps actually taken and reported zero fatal offsets in $2 \times 10^5$ events — survivorship bias in its textbook form, since a fatal offset is a death and never generates a step. Corrected instrumentation restored exact agreement with the uniform prediction.) A rigorous almost-all theorem needs one missing piece: the tail-hazard bound along orbits, an invariant-measure argument in the style of Tao's Collatz result — with the advantage that $R$ is explicit, total, piecewise affine, and its hazard is measured uniform.

## 4 The verification spectrum: where this sits, and the goal

It is useful to place this work on a spectrum of what "verifying a machine" can mean, because the endpoints are now both theorems.

*Starting verification* is the ground-up kind: replay and check every step. Our Lean development is this, industrialized — the kernel checks each of the $10^4$ startup steps individually, and the trust surface is two axioms and a proof checker. It is complete and unconditional, but it can never outrun the machine: verifying $n$ steps costs $n$.

*Predictive emulation* is the middle of the spectrum and the substance of this paper. Instead of replaying steps, certify the machine's *law* — the episode theorems and the five-integer map — and then verify *forecasts*: where the next probe event can occur, what the configuration will be $10^6$ steps from now, whether halting is reachable from the current shape. One certified episode theorem covers unboundedly many raw steps; the certified recurrence covers all of them. Verification cost decouples from runtime. This is what lets two machines' entire futures be checked against a conjecture a number theorist can hold.

*Ending verification* is the horizon, and we take it as the program's goal: $\mathsf{MIP}^* = \mathsf{RE}$ [10]. A classical polynomial-time verifier, interacting with two entangled provers, can verify *any* recursively enumerable statement — including "this Turing machine halts" for machines far beyond any budget — with the provers, not the verifier, bearing the unbounded cost. Non-halting statements such as ours live one level up (they are $\Pi_1$; zero-gap variants of $\mathsf{MIP}^*$ reach $\Pi_2^0$ [11]), so the entangled-prover frame in principle spans exactly the class of claims the busy beaver program generates. The

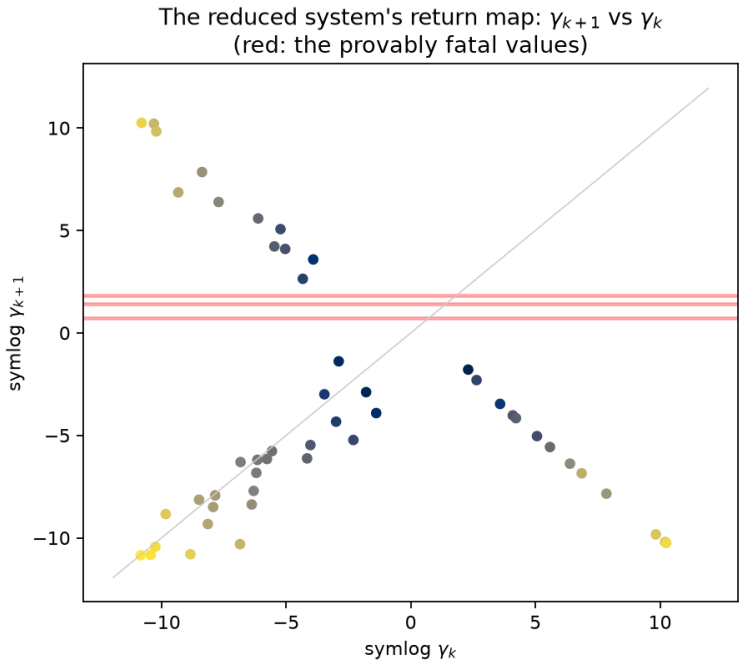



Figure 3: The reduced problem, as a dynamical system in its own right: the return map of the renewal invariant, $\gamma_{k+1}$ against $\gamma_k$ (symlog axes), over the first fifty renewals. The red bands are the values proven fatal. The conjecture: the map's orbit never lands in a band.

alignment we draw is this: kernel verification anchors truth at the bottom; predictive emulation compresses it in the middle; and $\mathsf{MIP}^* = \mathsf{RE}$ says that at the top, verification of halting-type claims is possible *in principle* with entanglement as the resource. Our quantum-inspired toolkit (reversible vector maps, state-space geometry) borrows disciplines from that endpoint; the open engineering-of-mathematics question is how much of the middle rung — certified forecasts — can be pushed toward it, replacing replay with interaction. Nothing in our results depends on this framing; it states the destination the pipeline is pointed at.

## 5 Why this may interest this community

Two reasons. The first is the workflow. This project is an instance of the emerging human–AI–kernel pattern in mathematics — AI systems draft and compile proofs at scale, a proof kernel arbitrates, humans own claims — the pattern of Tao's Equational Theories Project and the new Palomar registry for Lean-verified results [12, 13]. All of our Lean was drafted by an AI assistant from validated empirical specifications; the kernel checked roughly fifteen thousand lines; the generators are convenience, not trust surface. The second reason is methodological kinship. Finding that a high-dimensional trajectory is governed by a low-dimensional piecewise-affine law, by choosing coordinates in which the structure is linear, is the daily business of representation learning. Here the same move is carried one step further: the discovered representation is not evaluated on held-out data but proven exactly correct, step by step, against the system that generated it.

**Scope.** We claim no unconditional non-halting result. The coordinate charts guide discovery and exposition; no theorem depends on them. Empirical horizons are what we ran, not what is true.

**Acknowledgments.** The Lean developments, empirical validation, and proof generators were built in collaboration with an AI assistant (Claude, Anthropic). Every proof is checked by the Lean 4 kernel at the stated axioms.

## References

[1] The bbchallenge Collaboration. Determination of the fifth Busy Beaver value. arXiv:2509.12337.

[2] The bbchallenge Collaboration. Turing machine deciders, part I. arXiv:2504.20563.

[3] T. Tao. Almost all orbits of the Collatz map attain almost bounded values. arXiv:1909.03562.

[4] C. Moore. Generalized shifts: unpredictability and undecidability in dynamical systems. *Nonlinearity* 4 (1991).

[5] P. Kůrka. On topological dynamics of Turing machines. *Theoret. Comput. Sci.* 174 (1997).

[6] O. Viro. On basic concepts of tropical geometry. *Proc. Steklov Inst. Math.* 273 (2011).

[7] G. Litvinov. Maslov dequantization, idempotent and tropical mathematics. *J. Math. Sci.* 140 (2007).

[8] M. Nickel and D. Kiela. Poincaré embeddings for learning hierarchical representations. NeurIPS 2017.

[9] C. Series. The modular surface and continued fractions. *J. London Math. Soc.* 31 (1985).

[10] Z. Ji, A. Natarajan, T. Vidick, J. Wright, and H. Yuen. $\mathsf{MIP}^* = \mathsf{RE}$. arXiv:2001.04383; *Comm. ACM* 64(11), 2021.

[11] H. Mousavi, S. Nezhadi, and H. Yuen. Nonlocal games, compression theorems, and the arithmetical hierarchy. STOC 2022.

[12] T. Tao et al. The Equational Theories Project. arXiv:2512.07087.

[13] T. Tao. Palomar: a registry of Lean-verified mathematics, 2026. `palomar-registry.org`.